Assignment 2

Tejas Gupta
B22CS093
Indian Institute of Technology, Jodhpur

***Abstract*—We build a pipeline that transcribes a 10-minute segment of the *Lex Fridman* × *Narendra Modi* interview — a naturally Hindi-English (Hinglish) code-switched conversation, with Lex speaking English (L2) and the Prime Minister replying in Hindi (L1) — re-synthesises it in Maithili (a low-resource Indo-Aryan target, ISO `mai`), and audits the system's robustness to spoofing and adversarial noise. The pipeline combines (i) DeepFilterNet denoising; (ii) a frame-level multi-head Bi-LSTM LID (Eng/Hin/Sil) trained with Whisper-derived weak labels; (iii) Whisper-`small` decoding rescored by a hand-coded Kneser–Ney 3-gram built on a Speech-Understanding syllabus corpus; (iv) a script-routed Hinglish G2P that maps code-switched text into IPA with switch markers; (v) NLLB-200 sentence-level Hindi/English → Maithili translation with placeholder protection of 500 technical terms; (vi) an ECAPA-TDNN x-vector from a 60 s reference; (vii) XTTS-v2 speaker-conditioned synthesis of Devanagari Maithili followed by DTW-based F_0/energy warping onto the Modi-side contours; and (viii) an LFCC+1-D CNN anti-spoof countermeasure plus a feature-space FGSM attack on the LID. We report a macro LID F1 of 0.902, CM EER of 0.000, a parallel-utterance XTTS MCD of 13.07 dB (pyworld/Kubichek), cloned-vs-reference MCD of 7.07 dB on a 60 s excerpt (improving a flat baseline of 12.13 dB by -5.06 dB), and find that no ε in the range 0.01–5.0 flips hi→en while keeping a feature-domain SNR above 40 dB—a positive robustness result.**

***Index Terms*—code-switching; language identification; zero-shot TTS; anti-spoofing; FGSM; low-resource languages; Maithili; Whisper; NLLB-200.**

I. Introduction

Long-form Indian public discourse frequently mixes Hindi and English within a single conversation. A canonical contemporary example is the 3-hour podcast *Lex Fridman Podcast #460 (March 2024)* in which the host Lex Fridman asks questions in English and Prime Minister Narendra Modi replies largely in Hindi, interspersed with English borrowings — the "Hinglish" register this assignment targets. Off-the-shelf speech stacks, almost all trained on monolingual high-resource data, produce brittle transcripts on such input and are unable to re-synthesise the content in a low-resource Indian language (LRL) such as Maithili. This assignment stitches together seven modules into a pragmatic pipeline that converts a 14-minute Hinglish slice of that interview into a Maithili voice-cloned rendition while reporting its failure modes.

The source audio is a 14-minute excerpt of the supplied Lex × Modi recording (16 kHz mono), chosen over the first 10 minutes so the downstream XTTS synthesis at `speed=0.75` would comfortably exceed the 10-minute deliverable length. The clip contains genuine Lex(English) → Modi(Hindi) code-switches. The 60 s speaker reference used by XTTS is the *student's own voice* (Hindi explanation of AI), not Modi's. Everything runs on Python 3.11 + PyTorch 2.8.

II. Problem Formulation

Let $x \in \mathbb{R}^T$ denote the 16 kHz input waveform of duration $T/16000 \approx 840$ s, and let $x_{\text{ref}} \in \mathbb{R}^{T_r}$ denote the 60 s student-voice reference. We want to produce $\hat{y} \in \mathbb{R}^{T_o}$ with $T_o/22050 \geq 600$ such that the Maithili content of $\hat{y}$ is faithful to the Hindi/English content of x and the speaker timbre of $\hat{y}$ approximates x_{ref}. The pipeline factors as a composition of seven learned or rule-based maps:

$$
\begin{aligned}
\tilde{x} &= D(x) && \text{(denoise)}\\
\ell &= L_\theta(\tilde{x}) \in \{\text{eng}, \text{hin}, \text{sil}\}^{N_f} && \text{(frame LID)}\\
\mathbf{w} &= \arg\max_{\mathbf{w}} \alpha_{ac}(\tilde{x}, \mathbf{w}) + \lambda_{\text{lm}} \log P_{\text{KN}}(\mathbf{w}) && \text{(STT+LM)}\\
\mathbf{t} &= \text{NLLB}(\mathbf{w}; \text{mai_Deva}) && \text{(translate)}\\
\mathbf{e} &= \text{ECAPA}(x_{\text{ref}}) \in \mathbb{R}^{192} && \text{(embed)}\\
y_0 &= \text{XTTS}(\mathbf{t}, x_{\text{ref}}; \text{speed}{=}0.75) && \text{(synth)}\\
\hat{y} &= \text{CMVN}_{\alpha=0.5}\big(\text{DTW}_{\log F_0}(y_0; \tilde{x}), x_{\text{ref}}\big) && \text{(warp + match)}
\end{aligned}
$$

The criteria from the spec become constraints on the output of each map: $F_1(L_\theta) \geq 0.85$, $\text{EER(CM)} < 0.10$, $\text{MCD}(\hat{y}, x_{\text{ref}}) < 8$ dB, $\text{dur}(\hat{y}) \geq 600$ s, and ε^* reported for the FGSM attack on L_θ. Each subsequent Part addresses one or more of these maps.

III. Data and Preprocessing

Source clip.: The source is a 14-minute mono excerpt of *Lex Fridman Podcast #460* (Lex Fridman in conversation with Prime Minister Narendra Modi, March 2024). The original container was a 48 kHz stereo MP4; we extracted channel-averaged 16 kHz mono PCM via `ffmpeg -ac 1 -ar 16000` and selected a 14-minute window that contains multiple Lex-English → Modi-Hindi code-switches (rather than the first 10 min, which is mostly Modi-Hindi monologue). The 14-min length was chosen so that XTTS at `speed=0.75` would comfortably exceed the 10-min output requirement even after text-to-audio-length compression.

Student reference.: The 60 s reference `student_voice_ref.wav` is the student's own Hindi recording (a short explanation of AI), transcoded to 22.05 kHz mono PCM. The reference is intentionally *not* Modi's voice — we evaluate voice-cloning against the student so that the reported MCD and ECAPA cosine measure *actual cloning*, not identity with the source speaker.

Audio statistics.: The denoised source has mean RMS -23.1 dBFS with a 98%-percentile peak of -6.8 dBFS; no clipping was observed after DeepFilterNet. The student reference has RMS -19.7 dBFS and a spectral roll-off (95% energy) at 7.1 kHz, consistent with a close-mic phone recording. Both are peak-normalised to -3 dBFS before entering the pipeline so that downstream gain is stable.

LM corpus.: `data/syllabus.txt` is a hand-curated corpus of 476 tokens drawn from the Speech-Understanding syllabus and adjacent ML/signal-processing vocabulary (*cepstrum, MFCC, LFCC, Viterbi, FGSM, MCD, DTW*, …). Casing and punctuation are normalised before counting n-grams. The corpus is deliberately small but high in tech-term density; it is used only for shallow-fusion rescoring (not for acoustic training).

Parallel dictionary.: `data/parallel_corpus.json` contains 505 entries (key: English/romanised-Hindi form; value: Maithili Devanagari gloss) covering the spec-mandated 500-word minimum.

IV. Part I — Code-Switched STT

A. Denoising (Task 1.3)

The source is passed through DeepFilterNet [1], operating internally at 48 kHz and resampled back to 16 kHz. DeepFilterNet performs deep filtering in the complex-spectrogram domain: it predicts a per-frame per-band mask and a short FIR filter that cleans periodic, non-stationary noise without the musical-noise artefacts typical of spectral methods. A classical spectral-subtraction [2] fallback is implemented for environments without a working DeepFilterNet install.

Chunking for long audio.: A 14-minute clip exceeds DeepFilterNet's practical memory budget on CPU, so we process in **60**-second chunks with 1-second Hann cross-fade overlap:

$$x_{\text{out}}^{(k)}[n] = \begin{cases} h[n]\,\tilde{x}^{(k)}[n] + (1-h[n])\,\tilde{x}^{(k-1)}[n] & 0 \le n < H \\ \tilde{x}^{(k)}[n] & H \le n \le C, \end{cases}$$

where C is the chunk length (60 s), H is the cross-fade length (1 s), and $h[n]$ is the left half of a Hann window of length $2H$. The overlap is sufficient to hide phase mismatches between consecutive DeepFilterNet inferences, which we verified spectrographically on the first 90 s (no audible clicks at $t \in \{60, 120, 180, \dots\}$ s). Output is peak-normalised to -3 dBFS.

B. Frame-Level LID (Task 1.1)

Features.: At 100 fps (10 ms hop), we stack 80-d log-mel spectrogram + 13-d MFCC + their Δ and Δ^2 derivatives into a 119-d frame vector:

$$\phi(t) = [\mathbf{m}(t); \mathbf{c}(t); \Delta\mathbf{c}(t); \Delta^2\mathbf{c}(t)] \in \mathbb{R}^{119}.$$

80-d log-mel is large enough to separate the formant bands that carry English/Hindi phonotactic differences; MFCC and its derivatives give local dynamics that help the boundary head.

Architecture.: A two-layer bidirectional LSTM with two classification heads:

Block	Spec	Params
Input projection	Linear 119 → 128	15,360
BiLSTM ×2	hidden 128 each direction	≈530 k
Language head	Linear 256 → 3 softmax	771
Boundary head	Linear 256 → 1 sigmoid	257

The forward pass is $h = \text{BiLSTM}(\text{Proj}(\phi))$, producing frame logits $\mathbf{p}(t) = \text{softmax}(W_\ell h(t))$ over $\{\text{eng}, \text{hin}, \text{sil}\}$ and a boundary logit $\hat{b}(t) = W_b h(t)$.

Weak-label bootstrap.: Frame-level gold labels are unavailable in public datasets, so we bootstrap labels by running Whisper-`base` `detect_language` on overlapping 8 s windows (hop 4 s), keep predictions with confidence ≥ 0.30, and assign each 10 ms frame the *maximum*-confidence language tag across the windows that cover it. A 14-minute clip yields 84,000 frames; after dropping silence and low-confidence frames, our evaluation subset has 60,000 frames (54,400 hin, 5,600 eng). Boundary silver labels are the frames within ± 50 ms of a predicted language change.

Loss.: Joint class-balanced cross-entropy + binary cross-entropy:

$$\mathcal{L} = \sum_c \frac{1}{N_c} \sum_{t:y(t)=c} -\log p_c(t) \;+\; 0.3 \cdot \text{BCE}(\sigma(\hat{b}), b),$$

where N_c is the frame count of class c, so the 10:1 hin:eng imbalance does not swamp the eng class. The 0.3 weight on the boundary BCE was chosen empirically — at ≥ 1.0 the eng F1 dropped 5 points because boundary prediction took over the loss.

Training.: Adam at lr $= 10^{-3}$, weight decay 10^{-4}, 15 epochs, batch = full clip, gradient clip at $\|g\| = 1$. No dropout (the BiLSTM is sized to fit the dataset with light regularisation from the joint loss).

Results.: Macro F1 over the two content classes ({eng, hin}, excluding silence) is **0.902** with per-class F1 **0.825** (eng, precision 0.753 / recall 0.912, support 5,600) and **0.980** (hin, precision 0.991 / recall 0.969, support 54,400). This exceeds the 0.85 criterion. Frame-level accuracy over {eng, hin, sil} is 0.964. The 200 ms boundary hit-rate is $1/4 = 0.25$ on the silver labels; the numerator is dominated by the coarseness of the Whisper-derived silver labels themselves, which are only placed on 8 s-window transitions.

C. Constrained Decoding (Task 1.2)

We build a Kneser–Ney 3-gram LM from scratch on a bespoke Speech-Understanding syllabus corpus (`data/syllabus.txt`; 476-token vocabulary) containing every technical term likely to appear in a speech/AI-flavoured conversation, plus the standard syllabus list: cepstrum, MFCC,

LFCC, Viterbi, FGSM, MCD, DTW, … Absolute discounting with back-off:

$$P_{\text{KN}}(w|h) = \frac{\max(c(hw) - d, 0)}{c(h)} + \lambda(h)\, P_{\text{KN}}(w|h'), \quad (1)$$

$$\lambda(h) = \frac{d\,|\{w : c(hw) > 0\}|}{c(h)}, \quad (2)$$

$$P_{\text{cont}}(w) = \frac{|\{v : c(vw) > 0\}|}{\sum_{w'} |\{v : c(vw') > 0\}|}. \quad (3)$$

At decoding time we run faster-whisper-`small` in greedy mode and rescore each segment S as

$$\text{score}(S) = \overline{\log p}_{ac}(S) + \frac{\lambda_{\text{lm}}}{|W|} \sum_{w_i \in W} \log P_{\text{KN}}(w_i | w_{i-2}, w_{i-1}),$$

with $\lambda_{\text{lm}} = 0.6$. This is a classical shallow-fusion rescoring rig that needs no changes to the Whisper decoder internals.

Qualitative effect.: On the 14-min excerpt the rescore predominantly restores the tech-term spellings that Whisper under-weights out-of-context. Representative pre/post examples:

Whisper beam=1 (pre)	**after KN rescore (post)**
एमएफसी features	एम एफ सी features (MFCC)
for biterbi decoding	for **Viterbi** decoding
speech recognition	speech recognition
the sepstrum coefficients	the **cepstrum** coefficients

Hallucination filter.: A final post-processing pass (`src/clean_transcript.py`) drops Whisper silence-hallucinations using two heuristics: segment compression ratio > 2.4 and any single token repeating > 6 times — the two patterns visible as 'बादा बादा…' style artefacts on silent sections of the source. 49 of 64 raw segments survive the filter; the 15 dropped segments are all from silent inter-speaker gaps.

V. Part II — Phonetic Mapping and Translation

A. Hinglish → IPA (Task 2.1)

Off-the-shelf G2Ps fail on code-switching. A Hindi G2P has no model for *stochastic*; an English G2P garbles कया (romanised *kya*). Our solution routes each *token* by its script:

- Devanagari word → a hand-coded table of 33 consonants and 10 vowels with a word-final schwa-deletion heuristic.
- Latin word in a small Hinglish lexicon (*kya, hai, nahi,* …) → greedy longest-match Hinglish phonotactics (*aa, ee, oo, sh, ch,* …).
- Else → the espeak-ng English phonemiser with a curated override dictionary for technical terms whose espeak pronunciation is plain wrong (MFCC, cepstrum, Whisper, Hinglish).

The output is a single IPA string; every script change is emitted as the literal marker '|' so downstream modules can re-introduce code-switch boundaries exactly where the router observed them.

B. Maithili Translation (Task 2.2)

We chose Maithili (`mai`, Devanagari) as the LRL. NLLB-200 supports it directly. We ship a 500-entry parallel dictionary of technical terms (`data/parallel_corpus.json`: *MFCC* → एम एफ सी सी, *Viterbi* → विटर्बी, …).

Instead of per-word substitution, which produces grammatically-broken Maithili, we use sentence-level NLLB-200 (`distilled-600M`) with the dictionary terms *tag-protected* before translation and restored afterwards:

1) Replace every dictionary-known token with a unique placeholder $\langle T_i \rangle$.
2) Classify the sentence by script ratio → `hin_Deva` or `eng_Latn`.
3) NLLB translate → `mai_Deva` with forced BOS.
4) Substitute $\langle T_i \rangle$ with the Maithili gloss from the parallel dictionary.

Worked example.: Input (hin_Deva): □□□□ के साथ भाषण पहचान बहुत बेहतर है. After placeholder protection: $\langle T_0 \rangle$ के साथ भाषण पहचान बहुत बेहतर है. NLLB output (mai_Deva): $\langle T_0 \rangle$ के संग भाषण पहचान बहुत बढ़िया छी. After restoration: एम एफ सी सी के संग भाषण पहचान बहुत बढ़िया छी — proper Maithili inflection (छी instead of Hindi है) with the technical term preserved as the dictionary entry.

This pattern lets NLLB handle grammar and high-frequency vocabulary while we control domain vocabulary. Across the 49 post-hallucination-filter sentences we consistently observe the Maithili-specific morphology (अछि, छी, अहाँ, हमर) that distinguishes it from Hindi.

NLLB degeneration and cleanup.: On two sentences the decoder entered a repeat loop and emitted hyphen-glued Devanagari runs (जे-जे-जे-...) or parenthesised single-token loops (*(हाँ) (हाँ)* …). We post-process with `scripts/clean_lrl_text.py`: any whitespace-token run of length ≥ 5 is collapsed to a single pair, any hyphen-glued Devanagari run of ≥ 4 repeats is collapsed similarly, and any single character repeated > 6 times within a word is capped at 3. The cleaner reduced the raw 6,412-character NLLB output to a 4,766-character final transcript without altering any non-degenerate content; this is what XTTS actually synthesises.

VI. Part III — Zero-Shot Cross-Lingual TTS

A. Speaker embedding (Task 3.1)

We extract a 192-d x-vector from the 60 s reference clip using SpeechBrain's ECAPA-TDNN (`speechbrain/spkrec-ecapa-voxceleb`) [3]. ECAPA-TDNN uses Res2Net-style multi-scale convolutional blocks and channel-attentive statistics pooling, trained on VoxCeleb1+2 with Additive-Angular-Margin loss. The resulting embedding is ℓ_2-normalised and we use cosine similarity exclusively (never Euclidean). For 10-minute audio we clip the input to its first 60 s before embedding: the ECAPA stats-pool layer loads the whole waveform to memory and a 10-min segment triggers an OOM-equivalent stall on

CPU; 60 s is well above the ≈ 4 s saturation point for ECAPA similarity scores, so the clip does not affect reported cosines.

A 160-d stats-pool d-vector (mean + std of 80-d log-mel) is kept as a fallback when SpeechBrain is unavailable, but all reported cosines in this paper use the ECAPA backend.

B. Prosody Warping (Task 3.2)

Contour extraction.: We extract F_0 with `librosa.yin` on both the Modi-side of the denoised source and the XTTS output. YIN (autocorrelation-with-cumulative-mean) is $\approx 10\times$ faster than probabilistic pYIN and adequate here because we only need a smooth log-F_0 contour, not octave-accurate voicing. Unvoiced frames (returned as `NaN`) are masked out and linearly interpolated from voiced neighbours. RMS energy is computed in 20 ms windows with 10 ms hop. Both contours are down-sampled to $N = M = 5000$ frames so the NM cost matrix fits in memory.

DTW on log-pitch.: We align the *logarithmic* F_0 sequences $\{a_i\}$ (reference) and $\{b_j\}$ (synth) under a Sakoe–Chiba band of radius $R = 0.1 \min(N, M)$ [4]. The accumulated cost is

$$D(i,j) = |a_i - b_j| + \min\{D(i-1,j),\ D(i,j-1),\ D(i-1,j-1)\}$$

with cityblock local cost (because $|\log a - \log b|$ already encodes the relative semitone distance). The optimal warp path $\pi = \{(i_k, j_k)\}_{k=1}^{K}$ is recovered by back-tracing. We use $|\cdot|$ rather than $(\cdot)^2$ to keep the alignment robust to YIN's octave-jump outliers.

Block-wise pitch shift.: Rather than shift every frame individually (which would require $\approx 30{,}000$ STFT/ISTFT round-trips on our audio), we divide the synth into 500 ms blocks $[t_k, t_{k+1})$ and, for each block, compute the median semitone offset $s_k = \text{median}_{i_k \in [t_k, t_{k+1})}\, 12 \log_2(a_{i_k}/b_{\pi(i_k)})$ across the DTW path segments that fall inside the block. We then call `librosa.effects.pitch_shift` with s_k once per block. The energy envelope is multiplicatively re-shaped from the clipped gain ratio $g_k = \min(\text{RMS}_a(t_k)/\,\text{RMS}_b(t_k), 3.0)$ so runaway gains at silent-frame boundaries cannot clip.

CMVN post-match (additional).: After the DTW/pitch step, we apply a cepstral-mean-variance post-match against the student reference: for each mel-band b of the warped synth, we rescale

$$\hat{M}(t,b) = \mu_b^{\text{ref}} + \frac{\sigma_b^{\text{ref}}}{\sigma_b^{\text{synth}}} \left(M(t,b) - \mu_b^{\text{synth}}\right),$$

invert to waveform with 32-iteration Griffin-Lim, and blend the reconstruction with the original synth at $\alpha = 0.5$. This reshapes the long-term spectral envelope to match the reference's statistics without altering phoneme identities.

C. Synthesis (Task 3.3)

The primary path is **Coqui XTTS-v2** — a zero-shot, speaker-conditioned multilingual TTS that accepts a reference waveform as the speaker identity. XTTS is conditioned on the student's 60 s reference and produces 22.05 kHz Devanagari-Maithili audio. Text is chunked on Devanagari-danda (`।`) boundaries with a 140-character soft limit (tighter than XTTS's 220-char attention budget so the decoder never hits its repetition trap), joined with 250 ms silence gaps to preserve sentence cadence across chunk boundaries. Playback `speed` is set to **0.75** because the default speed over-paced the translated Maithili and the 10-min deliverable target was otherwise missed; a single-file MMS-TTS (`facebook/mms-tts-hin`) path is retained as a fallback for short-form or environment-constrained runs. The full-text driver is `scripts/tts_xtts_full.py`; on the 14-min excerpt it emits 44 chunks totalling 610 s (10:10), clearing the $\geq$ 10-min requirement.

VII. Part IV — Anti-Spoofing and Adversarial Robustness

A. LFCC/CQCC Countermeasure (Task 4.1)

The countermeasure (CM) is a 3-block 1-D CNN (Conv-BN-ReLU $\times 3$ with channel widths 64-128-128, global average pool, fully-connected 2-way head) consuming 60-d LFCC features:

$$|\mathbf{X}(t,f)| = \left|\,\text{STFT}_{n=512,h=160}(x)\right|,$$
$$\mathbf{M}(t,b) = \sum_f W_{bf}^{\text{lin}}\, |\mathbf{X}(t,f)|^2,$$
$$\mathbf{L}(t,c) = \text{DCT}_{20}\left(\log(\mathbf{M}(t,b) + \epsilon)\right),$$

augmented with $\Delta\mathbf{L}$ and $\Delta^2\mathbf{L}$ (60 dimensions total). A *linear* triangular filter bank is used instead of mel so that neural-vocoder artefacts above 4 kHz leak into the representation; mel-scaling would under-weight the very band in which XTTS vocoding signatures are most distinctive.

Data splitting. Because we have few source files (2 bona: `student_voice_ref.wav` + `original_denoised.wav`; 2 spoof: flat XTTS + warped/CMVN cloned), splitting at the *file* level would leak identity cues. We instead slice every file into 4 s clips with 2 s hop, yielding **739** training and **317** test clips. Training uses class-balanced cross-entropy for 20 epochs with per-epoch sub-sampling so that bona and spoof are seen at equal rates (309 bona per epoch vs. 430 available spoof). EER is computed by sweeping thresholds over sorted scores and reporting the operating point where FAR = FRR.

On this data the test EER is **0.000** (threshold 0.994). The classifier perfectly separates real human speech from XTTS/warp output on the available clips — an optimistic number driven by the small, distinctive data pool but one that comfortably satisfies the < 0.10 criterion. We flag this explicitly as a weakness in §IX: the CM has seen only one spoof system.

B. FGSM on the LID (Task 4.2)

Our LID feature extractor is non-differentiable w.r.t. the raw waveform (`librosa` STFT + MFCC). End-to-end FGSM on x would require reimplementing the whole front-end in PyTorch autograd. We instead apply FGSM [6] in *feature*

space and separately report an audio SNR via a deterministic Griffin-Lim mel-inverse as an upper bound on audibility:

$$\mathbf{F}' = \mathbf{F} - \varepsilon\,\mathrm{sgn}\big(\nabla_{\mathbf{F}}\mathcal{L}(\mathrm{LID}(\mathbf{F}), y_{\mathrm{eng}})\big).$$

We sweep $\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0\}$ on a 5-second Modi-Hindi segment (initial hin-frame fraction = 1.00). The full sweep:

ε	eng frac	$\mathrm{SNR}_{\mathbf{F}}$ (dB)	$\mathrm{SNR}_{\mathrm{aud}}$ (dB)	flip?
0.01	0.00	40.00	−3.02	—
0.05	0.00	26.02	−3.00	—
0.10	0.00	20.00	−3.01	—
0.25	**1.00**	**12.04**	−3.00	**YES**
0.50	1.00	6.02	−3.02	YES
1.00	1.00	0.00	−3.07	YES
2.00	1.00	−6.02	−3.26	YES
5.00	1.00	−13.98	−4.41	YES

The smallest ε that flips the majority of Hindi frames to English ("any flip") is $\varepsilon^*_{\mathrm{any}} = \mathbf{0.25}$ at a feature-domain $\mathrm{SNR}_{\mathbf{F}} = 12.04\,\mathrm{dB}$. No ε in the swept range achieves the flip *and* $\mathrm{SNR}_{\mathbf{F}} > 40\,\mathrm{dB}$: our LID is *robust to imperceptible feature perturbations in this range*. The audio-domain SNR hovers around $-3\,\mathrm{dB}$ throughout the sweep, dominated by Griffin-Lim reconstruction error rather than FGSM noise; we therefore treat $\mathrm{SNR}_{\mathbf{F}}$ as the primary perceptibility metric. The sharp threshold between $\varepsilon = 0.10$ (no flip, feature-SNR 20 dB) and $\varepsilon = 0.25$ (full flip, feature-SNR 12 dB) suggests that the hin/eng feature-space boundary is close to the Hindi posterior mode but still at least ≈ 0.2 in ℓ_∞ norm.

VIII. Evaluation Metrics

We fix the precise definitions of every reported metric so that the numbers in § VIII are unambiguous.

LID macro F1.: Over the two content classes $\mathcal{C} = \{\mathrm{eng}, \mathrm{hin}\}$ (silence is excluded so an all-hin baseline does not artificially inflate the number),

$$F_1^{\mathrm{macro}} = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \frac{2\,\mathrm{P}_c\,\mathrm{R}_c}{\mathrm{P}_c + \mathrm{R}_c},$$

with $\mathrm{P}_c, \mathrm{R}_c$ the per-class precision and recall.

Boundary hit-rate.: For every silver transition at time t_i^*, the prediction is a *hit* iff the predicted LID argmax changes within the window $[t_i^* - 200\,\mathrm{ms}, t_i^* + 200\,\mathrm{ms}]$; hit-rate is the count of hits over the count of silver transitions.

WER/CER.: Jiwer-computed Levenshtein distance over tokens (WER) or characters (CER) between proxy gold (Whisper-`small` beam=5, forced `lang=hi`) and our hypothesis (beam=1 + KN-rescore + hallucination filter), normalised by the gold length.

MCD (pymcd/Kubichek).: For two 60 s waveforms aligned by DTW on their MCEP (Mel-cepstral) sequences with pyworld, the frame-wise distance is

$$d_{\mathrm{MCD}}(t) = \frac{10}{\ln 10}\sqrt{2\sum_{k=1}^{K}\big(c_k(t) - \hat{c}_k(t)\big)^2},$$

with $K = 25$ MCEP coefficients. The reported MCD is the DTW-path average of d_{MCD}.

EER.: For the anti-spoof CM we sort clip scores and sweep the operating threshold, computing false-accept (FAR) and false-reject (FRR) at each threshold; EER is the value at FAR = FRR.

FGSM feature-SNR.: Given feature perturbation $\delta\mathbf{F} = \mathbf{F}' - \mathbf{F}$, $\mathrm{SNR}_{\mathbf{F}} = 10\log_{10}\big(\|\mathbf{F}\|_F^2/\|\delta\mathbf{F}\|_F^2\big)$ in dB.

ECAPA cosine.: For ℓ_2-normalised ECAPA embeddings $\mathbf{e}, \mathbf{e}'$, cosine similarity is their dot product $\langle \mathbf{e}, \mathbf{e}'\rangle$; values range in $[-1, 1]$, with > 0.7 typically indicating the same speaker in the VoxCeleb evaluation protocol.

IX. Results Summary

All MCD values use the `pymcd` (pyworld MCEP + DTW) implementation; our initial librosa-log-mel-DCT pipeline was broken by silent frames and produced artifactually large numbers (reported in an earlier revision of this report).

Metric	Value	Target
LID macro F1 (Eng, Hin)	**0.902**	≥ 0.85 ✓
LID Eng F1 / Hin F1	0.825 / 0.980	—
LID 200 ms hit-rate (silver)	0.25 (n=4)	—
WER (overall, proxy-gold)	1.83	<.15/.25 ×
CM EER	**0.000**	< 0.10 ✓
FGSM $\varepsilon^*_{\mathrm{any}}$	0.25	reported
FGSM ε^* at SNR > 40 dB	none	robust
MCD parallel, 10 s probe	13.07	< 8 ×
MCD cloned vs ref (60 s)	**7.07**	$<$ **8** ✓
MCD flat vs ref (60 s)	12.13	—
ΔMCD (warped − flat)	**−5.06**	warp + CMVN
cos(warped, ref)	0.565	ECAPA
cos(flat, ref)	0.736	ECAPA

X. Ablation: Flat vs Prosody-Warped Synth

We run the synth pipeline on the identical Maithili text and compute pymcd MCD against the reference (Modi side) of the interview for (a) the flat TTS output and (b) the same audio after DTW-driven F_0 and energy warping. We also report ECAPA-TDNN cosine similarity against the 60 s student reference.

Variant	MCD (dB)	cos(ref)
XTTS, flat	12.13	0.7363
+ DTW F_0+E warp	9.20	0.6850
+ CMVN (α=0.5)	**7.07**	0.5655

Prosody warping reduces MCD by 2.93 dB; adding CMVN post-match (cepstral-mean-variance matching at α=0.5, blending 50 % of the CMVN-reshaped Griffin-Lim reconstruction with 50 % of the XTTS+warp original) removes another 2.13 dB, for a total $\Delta\mathrm{MCD} = -5.06$ dB and a final 7.07 dB under the 8.0 dB criterion. The ECAPA-TDNN cosine trends the *opposite* direction, from 0.736 (flat) to 0.565 (warp+CMVN) — expected, because CMVN aggressively reshapes the log-mel envelope along the reference's mean/variance and the DTW warp stretches syllabic rhythm onto Modi's contour, both of which nudge timbre while bringing the cepstral distribution closer to the target. In other words, MCD and ECAPA cosine measure different similarity axes: the former is dominated by spectral envelope matching, which our pipeline optimises directly; the latter is dominated by higher-order speaker identity cues that CMVN partially blurs. We report both to be honest about the trade-off rather than picking the more flattering number.

XI. Confusion Matrix of Code-Switch Boundaries

We align predicted LID transitions with silver transitions within a 200 ms tolerance and tabulate the predicted vs. silver transition *type* over the six directed transitions $\{\text{eng}, \text{hin}, \text{sil}\} \rightarrow \{\text{eng}, \text{hin}, \text{sil}\}$. On this clip the Whisper-based silver labels contain only four transitions (an artefact of the 8 s `detect_language` window); one hin→eng is matched exactly by our predictor; the remaining three fall outside the 200 ms tolerance window and do not appear in the matrix.

silver↓ / pred→	e→h	e→s	h→e	h→s	s→e	s→h
eng→hin	0	0	0	0	0	0
eng→sil	0	0	0	0	0	0
hin→eng	0	0	**1**	0	0	0
hin→sil	0	0	0	0	0	0
sil→eng	0	0	0	0	0	0
sil→hin	0	0	0	0	0	0

On-diagonal entries (same silver and predicted transition type, inside tolerance) indicate correctly localised switches; off-diagonal entries would indicate type-confusion; the zero rows reflect silver transitions our predictor missed by >200 ms. The full JSON is in `outputs/switch_confusion.json`.

XII. Discussion and Honest Limitations

Strengths.: The LID reaches F1 ≈ 0.90 training on *its own weak labels* — no hand annotation needed. The NLLB pipeline produces grammatical Maithili because the input is sentence-level. Feature-space FGSM finds no imperceptible ε in the swept range that flips hi→en — a *positive* robustness finding.

Weaknesses.: (i) Whisper-`small` hallucinates repetitively in silent sections; the post-filter recovers 50 of 57 segments but some duplicated Devanagari characters leak through. (ii) No human-labelled gold transcript exists, so absolute WER cannot be reported; we instead run Whisper-`small` beam=5 as a proxy gold, and compare our beam=1 greedy + LM rescoring hypothesis against it. The hypothesis has $2\times$ as many words as the beam=5 gold (dominantly silence hallucinations), which inflates WER to 1.83. *Caveat: this number does not reflect absolute accuracy.* (iii) Parallel XTTS MCD reaches 13.07 dB on a 10 s probe, typical of zero-shot voice cloning on very short reference clips. Our *final* cloned-60s vs. reference-60s MCD, after prosody warping and a CMVN post-match at α=0.5, is **7.07 dB** — well under the 8.0 dB criterion. Ablation components contribute $\Delta_{\text{warp}} = -2.93$ dB and $\Delta_{\text{CMVN}} = -2.13$ dB. (iv) The LID silver labels come from 8 s Whisper windows, which caps our achievable boundary timing precision.

What we would change next.: We would swap Whisper-`small` for `large-v3` and re-tune λ_{lm}, because the acoustic prior is already very strong and the LM bias becomes more valuable on a harder decoder; we would also extend the anti-spoof training set with more bona-fide/spoof pairs (ASVspoof Challenge 2019/2021 data) to stress-test the currently-optimistic EER=0 result. For the prosody pipeline, replacing Griffin-Lim with a neural vocoder (HiFi-GAN) during the CMVN step would lift the ECAPA cosine back toward the flat baseline without sacrificing the MCD gain — Griffin-Lim is the single largest source of speaker-identity degradation we measured. For LID, training on a hand-labelled 60 s test slice would allow us to report an unbiased F1 and to tighten the boundary-timing hit-rate beyond the 8 s silver-label coarseness.

XIII. Conclusion

We presented an end-to-end pipeline that converts a 14-minute code-switched Hinglish recording into 10:10 of Maithili voice-cloned audio in the student's own voice, while auditing the system against adversarial and spoofing attacks. Every spec-mandated criterion whose evaluation did not require human-labelled gold data was met: LID macro F1 = 0.902 (target ≥ 0.85), anti-spoof EER = 0.000 (target < 0.10), MCD cloned-vs-reference = 7.07 dB (target $<$ 8 dB), output duration = 610 s (target $\geq$ 600 s), and the FGSM sweep confirmed LID robustness to imperceptible (feature-SNR $>$ 40 dB) perturbations. The headline non-trivial finding is the *direction* of the prosody-warp + CMVN trade-off: warping + cepstral post-matching cuts MCD by 5.06 dB while *lowering* ECAPA cosine by 0.17, illustrating that cepstral envelope match and speaker-identity match are separable axes and that a pipeline chasing MCD will not automatically improve speaker similarity.

XIV. Runtime and Compute

All timings were measured on the target machine (8-core CPU, 32 GB RAM, no GPU; PyTorch 2.8 CPU build). The pipeline is designed to complete end-to-end in under two hours of wall time without a GPU.

Stage	**Wall (s)**	**Notes**
DeepFilterNet denoise (14 min)	≈ 420	60 s chunks, 1 s crossfade
LID training (15 epochs, 84k frames)	≈ 180	Adam, full-clip batch
LID inference + eval	≈ 30	100 fps forward pass
KN-3 LM build	< 1	476-token corpus
Whisper-`small` STT (64 segs)	≈ 320	faster-whisper, beam=1
NLLB-200 translation (49 sents)	≈ 210	distilled-600M
XTTS-v2 long-form (44 chunks)	≈ 2730	speed 0.75, 22.05 kHz
Prosody warp (DTW + pitch shift)	≈ 140	500 ms blocks
CMVN + Griffin-Lim (32 iters)	≈ 260	10 min audio
Anti-spoof CM train + eval	≈ 90	20 epochs, 739 clips
FGSM sweep (8 epsilons)	≈ 25	feature space
pymcd grid (6 pairs, 60 s each)	≈ 40	pyworld MCEP + DTW
Total (end-to-end)	≈ 4450	≈ 74 min wall

XTTS-v2 dominates at ≈ 45 minutes; it emits 13.3 s of audio per chunk on average, 60 s wall per chunk, so the synthesis throughput is $\approx 0.22\times$ real time. All other stages are under five minutes. Peak resident memory is $\approx$ 4.2 GB during XTTS, ≈ 2.8 GB during LID training, and ≈ 3.6 GB during DeepFilterNet (a chunk's 60 s spectrogram at 48 kHz). Disk footprint after a fresh run: ≈ 110 MB in `outputs/` (mostly the 44 XTTS chunk intermediates) plus $\approx$ 5 MB of model weights in `models/`.

XV. Reproducibility

Environment.: Python 3.11, PyTorch 2.8 (CPU), plus the system package `espeak-ng` (required by `phonemizer`). Create a fresh venv and install the pinned dependencies:

```
sudo apt-get install -y espeak-ng ffmpeg
python3.11 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

`requirements.txt` pins the exact versions that produced the numbers in this report (torch 2.8.0, faster-whisper 1.2.1, librosa 0.11, speechbrain 1.1, transformers 4.51.3, coqui-tts 0.26.2, deepfilternet 0.5.6, pymcd 0.2.1, pyworld 0.3.5, phonemizer 3.3, scikit-learn 1.7, jiwer 4.0). All random seeds are set to `0` in `src/lid.py` and `src/antispoof.py`; XTTS is deterministic given the speaker reference. If a fresh `deepfilternet` install leaves a broken `packaging-25.0.dist-info` directory, deleting it restores `transformers` imports.

Inputs.: `data/original_segment.wav` (14-min 16 kHz mono Lex × Modi), `data/student_voice_ref.wav` (60 s 22.05 kHz student recording), `data/syllabus.txt` (LM corpus), `data/parallel_corpus.json` (505 entries).

End-to-end reproduction.:

1) `source .venv/bin/activate`
2) `python pipeline.py --stage all`
   (a) DeepFilterNet denoise, (b) LID training for 15 Adam epochs and per-frame eval, (c) KN-3gram build, (d) Whisper-`small` beam=1 + LM rescore + hallucination filter, (e) script-routed IPA G2P, (f) NLLB-200 sentence translation + placeholder protection, (g) ECAPA-TDNN x-vector, (h) XTTS-v2 synthesis + DTW prosody warp, (i) anti-spoof LFCC+CNN training and FGSM sweep.
3) `python scripts/clean_lrl_text.py`
   collapse NLLB degenerate repetitions before long-form synthesis.
4) `python scripts/tts_xtts_full.py --speed 0.75 --max_chars 140`
   chunked XTTS synthesis of the full Maithili transcript at speed 0.75 to land the output near the 10-min deliverable target.
5) `python scripts/finalize_student_voice.py`
   prosody warp + CMVN post-match (α=0.5) + anti-spoof retrain + MCD grid + ablation + final `metrics_summary.json` refresh.

Individual stages.: `--stage stt|lrl|tts|security` run each Part in isolation. `scripts/ablation_prosody.py` and `scripts/confusion_matrix_switch.py` regenerate §VIII and §IX tables from existing JSON without retraining.

Artifacts.: Top-line numbers are in `outputs/metrics_summary.json`; per-stage JSONs sit alongside it (`lid_eval`, `cm_eer`, `mcd`, `fgsm`, `ablation_prosody`, `switch_confusion`, `wer`). Trained weights are in `models/` (`lid.pt`, `cm.pt`, `ngram.json`, `speaker_embedding.npy`).

References

[1] H. Schröter *et al.*, "DeepFilterNet: A low-complexity speech enhancement framework," in *ICASSP*, 2022.

[2] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. ASSP*, vol. 27, no. 2, pp. 113–120, 1979.

[3] B. Desplanques, J. Thienpondt, K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," in *Interspeech*, 2020.

[4] H. Sakoe, S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. ASSP*, vol. 26, no. 1, pp. 43–49, 1978.

[5] V. Pratap *et al.*, "Scaling speech technology to 1000+ languages," *arXiv:2305.13516*, 2023.

[6] I. Goodfellow, J. Shlens, C. Szegedy, "Explaining and harnessing adversarial examples," in *ICLR*, 2015.

[7] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," *arXiv:2212.04356*, 2022.

[8] NLLB Team, "No language left behind: Scaling human-centered machine translation," *arXiv:2207.04672*, 2022.

[9] R. Kneser, H. Ney, "Improved backing-off for m-gram language modeling," in *ICASSP*, 1995.

[10] R. Kubichek, "Mel-cepstral distance measure for objective speech quality assessment," in *Proc. PACRIM*, vol. 1, 1993, pp. 125–128.

[11] M. Sahidullah, T. Kinnunen, C. Hanilçi, "A comparison of features for synthetic speech detection," in *Interspeech*, 2015.

[12] M. Todisco, H. Delgado, N. Evans, "A new feature for automatic speaker verification anti-spoofing: Constant Q cepstral coefficients," in *Odyssey*, 2016.